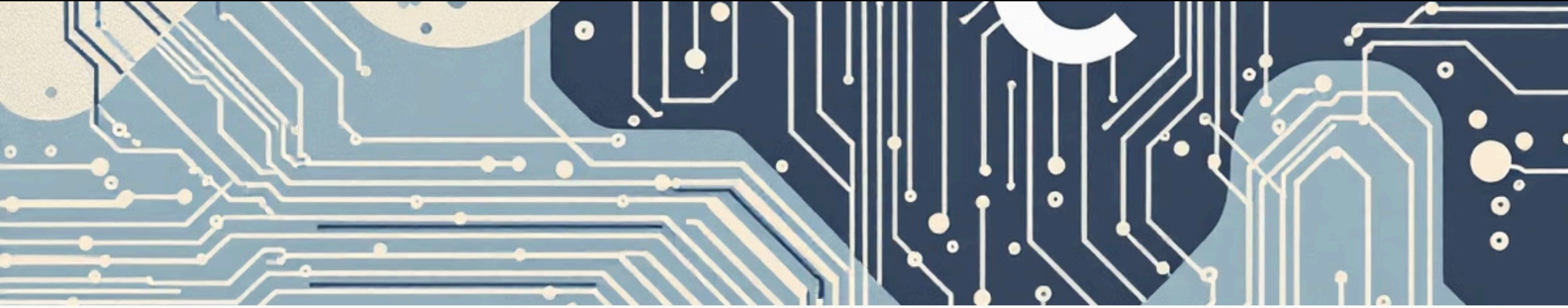
Newton–Raphson Method in C using Python

A Project Presentation by Aromal Madakavil



Introduction

This project implements the Newton–Raphson method in C while leveraging Python's SymPy library for symbolic differentiation. It elegantly combines the computational efficiency of C with the powerful mathematical capabilities of Python, showcasing effective interoperability between compiled and interpreted languages.

Objectives

1

Understand & Implement

Master the theoretical and practical aspects of the Newton–Raphson method.

2

Embed Python in C

Utilize the Python/C API (Python.h) to integrate Python functionalities within C code.

3

SymPy for Differentiation

Harness Python's SymPy library for automated symbolic derivative computations.

4

Dynamic Evaluation

Develop mechanisms to dynamically evaluate mathematical expressions at runtime.

5

C–Python Interoperability

Explore the nuances and benefits of seamless communication between C and Python.

Working Principle

01

Function Input

User defines the mathematical function, e.g., `x**2 - 3*x + 2`.

02

Python (SymPy) Differentiation

SymPy computes the symbolic derivative $f'(x)$ of the input function.

03

C String Retrieval

The C program retrieves both the original function $f(x)$ and its derivative $f'(x)$ as text strings.

04

TinyExpr Evaluation

TinyExpr dynamically evaluates $f(x)$ and $f'(x)$ numerically for each iteration.

05

Newton–Raphson Application

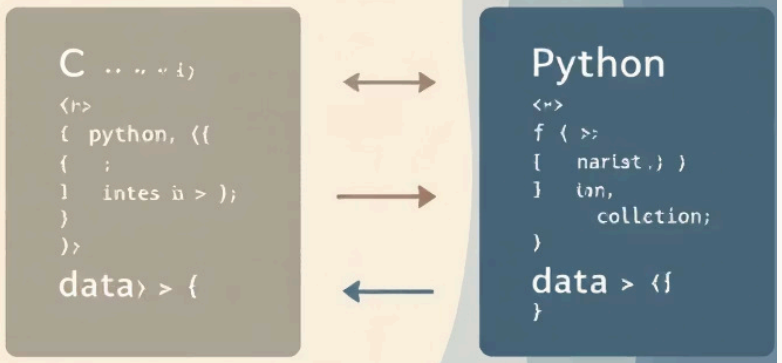
The Newton–Raphson formula is applied:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

06

Convergence Check

The iterative process continues until the desired level of convergence or tolerance is achieved.



Code Overview

- **Initialization:** Uses `Py_Initialize()` to start the embedded Python interpreter within the C program.
- **SymPy Execution:** Executes Python (SymPy) commands for symbolic differentiation using `PyRun_SimpleString()`.
- **Data Exchange:** Retrieves the computed derivative expression from Python back into the C environment.
- **Expression Evaluation:** Employs the TinyExpr library in C for efficient numerical evaluation of `f(x)` and `f'(x)`.
- **Root Approximation:** Implements the Newton–Raphson iterative algorithm to approximate the root of the given function.

Python is embedded within C using `Py_Initialize()` and `PyRun_SimpleString()` to compute the derivative symbolically.

```
Py_Initialize();
PyRun_SimpleString(
"import sys\n"
"import sympy\n"
);
```

```
printf("\nf(x) = %s", func);
printf("\nf'(x) = %s\n", fprime_str);
printf("\nPython derivative done. Starting C iteration\n");
Py_Finalize();
```

TinyExpr compiles and evaluates both $f(x)$ and $f'(x)$ numerically.

```
    for (int i = 0; func[i]; i++) {   //loop because tinyexpr takes ** as ^
        if (func[i] == '*' && func[i+1] == '*') {
            func[i] = '^';
            memmove(&func[i+1], &func[i+2], strlen(&func[i+2]) + 1);
        }
    }
    for (int i = 0; fprime_str[i]; i++) {
        if (fprime_str[i] == '*' && fprime_str[i+1] == '*') {
            fprime_str[i] = '^';
            memmove(&fprime_str[i+1], &fprime_str[i+2], strlen(&fprime_str[
                i+2]) + 1);
        }
    }
```

```
printf("\nNewton-Raphson Iterations:\n");
double fx, fpx;
te_expr *expr_f, *expr_fp;
//compiled expressional object representing f(x)
te_variable vars[] = { {"x", &x0} };
// defines the variable x in the expression and points the value of x0
```

```
do {
   expr_f = te_compile(func, vars, 1, 0);
   //{te_compile(string func , variable arr , no. of variables,
      pointer to an error position)}
   fx = te_eval(expr_f);
   //evaluates the function with the value of x0
   te_free(expr_f);
   //frees the memory used by expr_f
   expr_fp = te_compile(fprime_str, vars, 1, 0);
   fpx = te_eval(expr_fp);
   te_free(expr_fp);
```

```
    if (fpx == 0) {
        printf("Derivative zero at x = %lf\n", x0);
        break;
    }
    x1 = x0 - fx / fpx;
    printf("Iter %2d: x = %.10lf\n", iteration + 1, x1);
    if (fabs(x1 - x0) < tolerance) {
        printf("Converged within tolerance after %d iterations.\n",
            iteration + 1);
        break;
    }
    x0 = x1;
    iteration++;
} while (iteration < 100);
printf("\nApproximate Root = %lf\n", x1);
```

Key Learnings

Python–C Linking

Proficiently linking and embedding Python directly into C programs.

Symbolic Computation

Mastery of symbolic computation with Python's powerful SymPy library.

Numerical Evaluation

Effective numerical evaluation of expressions using TinyExpr in C.

Resource Management

Handling Python objects, memory, and variable management across language boundaries.

Numerical Algorithms

Deepened understanding of numerical root-finding algorithms and their practical application.

Challenges Faced

Linking Libraries

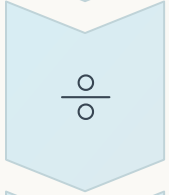
Successfully configuring and linking Python headers and libraries with GCC compilers.

Syntax Translation

Converting Python's exponentiation operator (`**`) to TinyExpr's (`^`).

Data Exchange

Seamlessly managing data and object transfer between the Python and C environments.



Zero Division

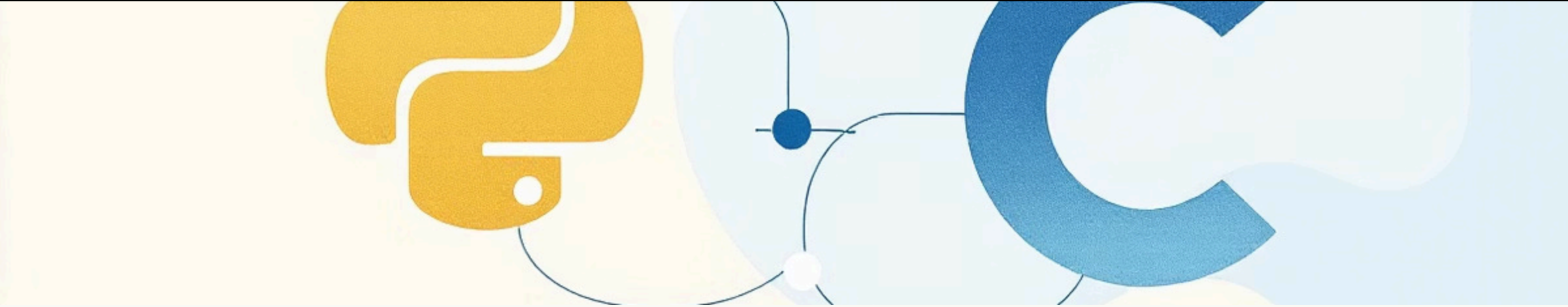
Implementing robust error handling for division by zero scenarios when `f'(x) = 0`.

Runtime Debugging

Diagnosing and resolving complex integration issues that arose during runtime.

Outcome

- **Successful Integration:** The program flawlessly integrates C and Python, demonstrating effective hybrid computation capabilities.
- **Automated Differentiation:** Achieved automated derivative generation using SymPy, enhancing the efficiency of the root-finding process.
- **Accurate Root Finding:** The implemented Newton–Raphson method consistently performs accurate root approximation.
- **Enhanced Understanding:** The project significantly deepened understanding of numerical algorithms and the intricacies of language interoperability.



Conclusion

This project successfully demonstrates how combining C and Python enhances computational flexibility and power. The hybrid approach strikes an optimal balance between the high execution speed of C and the symbolic manipulation capabilities of Python, offering a robust solution for numerical problems.

Thank you!